Futrix AI: Complete Architecture, Engineering and Product Blueprint

AI Career Twin and ATS Resume Diagnostic Platform | Version 2.0.0 | October 2026

1. Executive summary

Futrix AI is an **AI career twin and resume diagnostic platform**. A candidate pastes a resume and a target job description, and the platform returns a transparent ATS score from 0 to 100, a breakdown of what was checked, the matched and missing skills, and concrete fixes. The score is shown on an interactive 3D keycap orb rendered in raw WebGL2, and every analysis is saved so the candidate can watch their score and skills change over time.

Its main feature is the 50-point rejection gate: any resume that scores below 50 is flagged as **High Rejection Risk** with a crimson warning, because it is likely to be filtered out by automated screening before a human reads it.

At a glance

Item	Value
Architecture	Four layers: React client, Node.js gateway, Python AI engine, Supabase PostgreSQL
Scoring	Deterministic, 4 weighted components plus a mismatch penalty, no LLM in the scoring path
Signature feature	Hard rejection gate at a score under 50
Visual identity	Self-contained WebGL2 keycap orb, no Three.js, frontend under 135 kB gzipped
Privacy	Private PostgreSQL, per-user data isolation, no resale or model training on resumes
Quality	105 automated tests reported passing (74 Python, 31 Node)

2. The problem and the solution

The problem

Most large employers receive far more applications than recruiters can read, so applicant tracking systems (ATS) such as Workday, Taleo and Greenhouse sort and filter candidates first. Candidates who are filtered out usually get an automated rejection with no explanation. Many commercial resume checkers make this worse by inflating scores so that a flawed resume looks healthy, which pushes users toward paid plans and sends weak resumes into real applications.

The Futrix AI answer

1. **A living career twin.** Instead of a one-time scan, Futrix keeps a persistent record of the candidate's skills, scores and gaps, and shows the change between versions (for example, a gain of 17 points).
2. **A tactile 3D interface.** The real ATS score is engraved on every key of a 3D keycap orb that reacts to dragging, clicking and typing with ripple waves.
3. **An honest rejection rule.** Scores under 50 are never softened. The user sees an immediate warning before spending an application.
4. **Transparent math.** Five checks produce the score using published weights, so the user can see why a resume lost points.
5. **Private by design.** Resume text stays in a private database and is not used to train public models.

Four mental models

Concept	Analogy	What it explains
ATS	Airport luggage scanner	A smudged or twisted barcode gets rejected by the machine, so unreadable resume layouts get rejected by software
Under-50 rule	Expired passport	Border control does not give 70% credit, and a failing resume should not get flattering partial praise
Career twin	Fitness tracker	Continuous vital signs and trends, not a single check-up

Concept	Analogy	What it explains
Futrix AI	Flight simulator	A safe place to stress-test a resume before the real flight

3. What makes Futrix AI different

Area	Typical resume checker	Futrix AI
Model of the user	One-time transaction	Persistent career twin with score history and skill deltas
Scoring	Opaque, often inflated	Open formula with explicit weights and a hard rejection gate
Interface	Flat PDF report or progress ring	GPU-rendered 3D orb with the score engraved on every key
Parsing checks	Keyword counts	Sections, date parseability, encoding hygiene, lexical match, prerequisite penalty
Data handling	Often shared with third-party AI providers	Private PostgreSQL with strict per-user isolation
Bundle cost	Heavy 3D or chart libraries	Raw WebGL2 shaders, under 135 kB gzipped, no Three.js (about 600 kB avoided)

Four core innovations

1. **Career twin paradigm.** The system retains chronological history, computes gains across revisions, and tracks readiness for target roles.
2. **Proprietary WebGL2 keycap orb.** Procedural keycap geometry with tapered bodies, bevels and dished tops, a runtime canvas texture atlas for the numbers, instanced draw calls, momentum drag and ripple physics.
3. **Anti-inflation fairness filter.** The under-50 rule is the product's honesty guarantee.
4. **Sovereign data architecture.** Isolated services, a private database, hashed refresh tokens and no resume resale.

4. The 50-point rejection gate (main feature)

Every analysis ends with a gate decision. A composite score under 50 sets `isSevereMismatch = true` and `statusLabel = High Rejection Risk`, the orb floor glow turns crimson rose (`#f43f5e`), and the page shows the root causes.

Tier	Score	Meaning	Visual
1. Fast-track	80 to 100	Clean format, valid dates, strong overlap with the job requirements	Blue or silver glow (`#3b82f6`)
2. Competitive	60 to 79	Passes screening cleanly, minor secondary skill gaps	Standard glow
3. Vulnerable	50 to 59	Parseable but borderline, at risk when applicant volume is high	Amber caution
4. Straight rejection	Under 50	High Rejection Risk, treated as automatically disqualified	Crimson rose (`#f43f5e`)

Root causes that push a resume under 50

- Missing mandatory section headers, such as no Experience or no Education section detected.
- Encoding corruption: multi-column reading inversions, broken font ligatures, a non-ASCII ratio above 10%, or replacement characters (`\uFFFD`).
- Dates that cannot be parsed into a timeline.
- Almost no core prerequisite skills (lexical match under 20%).

How to read the gate honestly. Real employer ATS products do not publish a universal score cutoff, so the 50 mark is **Futrix's own risk threshold**, built to reflect common failure patterns. It is a conservative warning, not a guarantee that a specific company will reject the resume.

5. System architecture

Futrix AI is a decoupled monorepo with four layers and a target latency of under 90 ms for the analysis call.

Layer	Technology	Responsibility	Default port
Client	React 18, Vite 5, TypeScript, Material UI, raw WebGL2	Input forms, results, 3D orb, career twin views	5173
Gateway	Node.js, Express 5, JWT, Google Identity Services	Authentication, rate limiting, request proxy, persistence	5000
AI engine	Python, FastAPI, Uvicorn	Text normalization, five-stage heuristic scoring	8000
Database	Supabase PostgreSQL 15	Users, analyses, refresh tokens, JSONB payloads	PostgreSQL wire

The client talks to the gateway over REST and JSON. The gateway proxies analysis requests to the AI engine and writes results to PostgreSQL. The AI engine never touches the database directly.

6. Technology stack

Versions are as recorded in the project specification.

Layer	Technology	Version	Why it was chosen
Frontend framework	React	18.3.1	Declarative components, stable concurrent rendering, mature ecosystem
Build tool	Vite	5.4.0	Fast hot reload, Rollup bundles under 135 kB gzipped
Language	TypeScript	5.5.3	Strict typing with `tsc -b`
UI system	Material UI	7.3.8	Accessible primitives and frosted-glass styling
3D	Raw WebGL2 (GLSL 300 es)	n/a	Custom shaders and instanced geometry with zero 3D library dependency
Client state	Zustand	5.0.12	Small atomic store without Redux boilerplate

Layer	Technology	Version	Why it was chosen
Icons	Lucide React	1.59.0	Clean SVG glyphs (confirm this version string against the package registry)
API gateway	Express	5.2.1	Native async error handling, low memory footprint
Sign-in	Google Identity Services	v3	Client-side OAuth 2.0 token flow, no server-side secret exposure
Sessions	JSON Web Tokens	9.0.3	15-minute access token plus 7-day rotated refresh token
AI framework	FastAPI	0.110+	Async Python with Pydantic validation
ASGI server	Uvicorn	0.28+	uvloop and httptools speed
Database	Supabase PostgreSQL	15.x	Relational integrity, JSONB indexing, row-level security, backups
Testing	Pytest	9.1.1	74 Python assertions

7. How it works, step by step

1. The candidate enters resume text and a target job description.
2. The client validates a minimum of 50 characters and calls the check endpoint.
3. The gateway applies a rate limit of 5 audits per minute per IP or user.
4. The gateway proxies the payload to the Python AI engine.
5. The engine runs the five-stage pipeline described below.
6. The engine returns `ats_score`, per-check results and suggestions.
7. The gateway saves the result to the `analyses` table.
8. The gateway returns the response to the client, targeting under 90 ms.
9. The client engraves the score on the orb, sets the floor glow, renders the matched and missing keyword chips, and updates the career twin history.

The five-stage engine

Stage	Name	What it does
A	Normalize	Normalizes unicode, strips non-ASCII corruption, repairs hyphenated line breaks, produces a token array
B	Sections	Detects Summary, Experience, Education, Skills and contact sections through weighted pattern matching over 100+ header variations
C	Dates	Scans for `MM/YYYY`, `Month YYYY` and `YYYY - Present` patterns and flags vague or missing durations
D	Lexical match	Compares resume tokens with the job requirements and a taxonomy of 1000+ technology skills using N-gram density
E	Gate	Applies the weights, the mismatch penalty and the under-50 rule

8. Scoring model

```
S_ATS = clamp( 0.40 * K_match + 0.25 * S_pres + 0.15 * D_parse + 0.20 *
F_clean - P_mismatch , 0, 100 )
```

Component	Weight	What it measures
K_match	40%	Lexical and N-gram overlap between resume and job requirements
S_pres	25%	Presence of standard sections
D_parse	15%	Share of dates that parse unambiguously
F_clean	20%	Encoding hygiene: ASCII purity, non-ASCII ratio under 5%, no replacement characters
P_mismatch	up to minus 35	Penalty for Senior, Lead or Architect targets when fundamental prerequisites are missing

Section presence weights (S_pres): work experience 35%, contact information 20%, professional summary 15%, education and certifications 15%, skills matrix 15%.

Design properties: the function is deterministic, so the same input always gives the same score, and every point can be traced to a named check.

9. Data model

Table	Key fields	Notes
`users`	`id` (UUID), `email` (unique), `name`, `auth_provider`, `avatar_url`, `login_attempts`, `lock_until`, timestamps	Unique email index for fast lookup; attempt counter and lock time defend against credential stuffing
`analyses`	`id`, `user_id`, `resume_text`, `target_role`, `readiness_score`, `ats_score`, JSONB `skills`, `gaps`, `recommendations`, `metadata`, `created_at`	Compound index on user and creation time; history queries skip `resume_text`, shrinking payloads by up to 94%
`refresh_tokens`	`id`, `user_id`, `token_hash` (unique), `is_revoked`, `expires_at`, `created_at`	Stores SHA-256 hashes of single-use tokens; supports immediate revocation when reuse is detected

One user owns many analyses and many refresh tokens.

10. Authentication and security

1. The user signs in with Google Identity Services and receives an OAuth access token.
2. The client posts the token to `/api/auth/google`.
3. The gateway verifies it by calling Google's userinfo endpoint and receives email, name, picture and subject.
4. The gateway finds the user by email or creates a new record with `auth_provider = google`.
5. The gateway issues a 15-minute access token and a 7-day refresh token and stores the refresh token's SHA-256 hash.
6. The client receives the token pair and the user object.

Security controls: hashed and rotated refresh tokens, revocation on reuse, rate limiting at 5 audits per minute, login lockout fields, row-level security and per-user data isolation (IDOR protection), and no use of resume data for public model training.

11. The 3D keycap orb engine

The orb lives in `client/src/components/ui/keycap-orb.tsx`.

- **No 3D library.** Raw WebGL2 with GLSL 300 es shaders keeps the bundle small.
- **Procedural geometry.** Tapered keycap bodies, roll-over bevels and spherical top dishes.
- **Runtime texture atlas.** Numbers and glyphs are painted on an in-memory canvas and bound as a texture, so no font or image downloads are needed.
- **Instanced rendering.** Up to 1,500 keycaps plus the core are drawn with instanced calls at 60 fps.
- **Materials.** Opaque plastics (Ink, Ash, Cobalt, Lime) and clear glass computed with the Fresnel term `F = 0.04 + 0.96 * (1 - N.V)^5`, which reveals the cross-stem.
- **Interaction.** Ray-box and ray-sphere picking, drag with momentum, and exponential ripple waves on click or typing.
- **Battery saving.** An `IntersectionObserver` stops animation when the orb scrolls out of view.
- **Score mapping.** Each key's label is the score, and the floor glow is `#3b82f6` for a pass and `#f43f5e` for the under-50 gate.

12. Competitive positioning

Competitor	How it works	Weakness cited	Futrix AI response
Jobscan	Keyword density matching	Paywalled scans and an incentive to stuff keywords	Free, natural lexical density without stuffing
Resume Worded	Static report	Opaque scoring and generic advice	Open formula with an interactive 3D result
Teal	Application tracker	Shallow ATS analysis	Deeper checks for encoding, dates and seniority
ChatGPT or Claude prompts	Conversational review	Cannot reliably parse PDF layout; privacy depends on settings	Deterministic engine and a private database
Enterprise ATS	Corporate filtering	No transparency to candidates	Pre-application simulation of common filter failures

The competitor weaknesses above come from the project specification and should be re-verified (pricing and features change) before external publication.

13. Who it is for

Audience	Value
Competitive tech job seekers	Confirms that technical keywords are indexed and parseable
University career centers and bootcamps	Objective, repeatable resume audits that replace manual reviews
Career changers and self-taught developers	Shows missing prerequisite skills before applying
Recruiters and talent consultants	Spots formatting risks that make candidates disappear in screening

14. Quality and testing

Run everything from the repository root with `npm test`.

Layer	Framework	Result	Time
Python AI engine	pytest 9.1	74 of 74 passed	0.52 s
Database and security	Node.js test runner	31 of 31 passed	4.38 s
Type check and build	`tsc -b && vite build`	1,050 modules, 0 errors	9.61 s
Lint	ESLint 9	0 errors	3.52 s
Total	Monorepo suite	105 tests passed	under 15 s

These figures are taken from the project specification as reported.

15. Claims to validate before investor or public use

The following statements are strong and should be backed by evidence or reworded.

Statement	Concern	Suggested handling
"Over 75% of qualified resumes are discarded by ATS before a human reads them"	Widely repeated but I could not trace it to a reliable primary source	Cite a source or say "many"
"Under 50 means straight rejection by corporate ATS"	Real ATS products have no universal cutoff; 50 is Futrix's own threshold	Say "high rejection risk under Futrix's screening-risk model"
"Zero LLM hallucinations"	True for the scoring path only if no LLM generates the score or advice text	State precisely where an LLM is or is not used
"Directly simulates Workday, Taleo and Greenhouse rules"	Those vendors do not publish their rules	Say "simulates common parsing and screening failure patterns"
Competitor pricing and feature claims	Change often	Re-check and date-stamp them

Statement	Concern	Suggested handling
"Verified production release"	Needs a deployment and monitoring reference	Link release notes and uptime data
Lucide React 1.59.0	The version string looks unusual	Confirm in the package registry
Latency target of under 90 ms	Needs a measurement	Publish a benchmark with p50 and p95

16. Instructions for AI assistants using this document

1. For slides, lead with the four-layer architecture, the airport scanner and expired passport analogies, and the raw WebGL2 orb under 135 kB.
2. For competitive or pitch material, contrast deterministic scoring with chatbot advice, stress the private database, and cite the 105 passing tests, while applying the validation table in section 15.
3. For code changes, keep the monochromatic palette (`#0a0a0a` obsidian, white, silver and cobalt glows), preserve the under-50 gate, and keep both `id` and `_id` on API payloads for backward compatibility.